Alternative Assessment Tool(AAT) Report on

“Simulation of sequential and combinational circuit in cadence tool ”

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT-2) for the course

ADVANCED DIGITAL LOGIC DESIGN

[Course code: 23EC5PE1AD]

SUBMITTEDBY

PARINEETHA U 1BM25EC431

Course Instructor

Prof. Ajaykumar Devarapalli

Department of Electronics and Communications

BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year **2026 – 2027**

TABLE OF CONTENTS

Sl. No.	Experiment Title	Page No.
	Combinational Circuit	
1	4 Bit Comparator	3-5
	Sequential Circuit	
2	Ring Counter	6-7

Experiment 1(Combinational Circuit) -4bit Comparator

Procedure:

1. Create folders with names “RTL” , “TB” and “RESULT” to store the code, testbench and simulation results respectively.
2. Open the terminal.
3. Open the folder RTL using cd command.
4. Use “gedit compare_4.v” to open and edit a Verilog file, add your code her and save it.
5. To check for errors, close the code file and in the terminal type “ncverilog +access+r compare_4.v”.
6. To open folder “TB” type “ cd ../TB”.
7. And type “gedit tb_compare.v” to open and edit a Verilog file, add your testbench code here.
8. Close the testbench code file and type “cd ../RESULT” to open the result folder.
9. To execute the tetsbench and code type “ncverilog +access+r ../RTL/compare_4.v ../TB/tb_compare.v”.
10. This generates the vcd file in RESULT folder.
11. To check if the vcd file is present in RESULT folder enter “ls”.
12. For simulation we have used simvision.
13. To open the simulation enter simvision compare.vcd.

Code:

```
module compare_4 (
   input [3:0] A,
input [3:0] B,
output reg GT,
output reg EQ,
   output reg LT
);

always @(*) begin
   GT = 0;
   EQ = 0;
   LT = 0;

   if (A > B)
GT = 1;     else if
(A == B)        EQ
```

```
= 1;     else
LT = 1; end

endmodule
```

Testbench Code:

```
module tb_compare_4;

   reg [3:0] A;
   reg [3:0] B;

   wire GT;
wire EQ;
   wire LT;

   // Instantiate the Design Under Test (DUT)
compare_4 uut (
       .A(A),
       .B(B),
       .GT(GT),
       .EQ(EQ),
       .LT(LT)
   );

   initial begin
       $display("Time\t A\t B\t GT\t EQ\t LT");
       $monitor("%0t\t %d\t %d\t %b\t %b\t %b",
                $time, A, B, GT, EQ, LT);

       // Test case 1: A > B
A = 4'd10; B = 4'd5;
       #10;

       // Test case 2: A == B
       A = 4'd7; B = 4'd7;
       #10;

       // Test case 3: A < B
       A = 4'd3; B = 4'd9;
       #10;

       // Test case 4: A = maximum, B = minimum
       A = 4'd15; B = 4'd0;
```

```
        #10;

        // Test case 5: A = minimum, B = maximum
        A = 4'd0; B = 4'd15;
        #10;

        // Test case 6: Both inputs are zero
        A = 4'd0; B = 4'd0;
        #10;

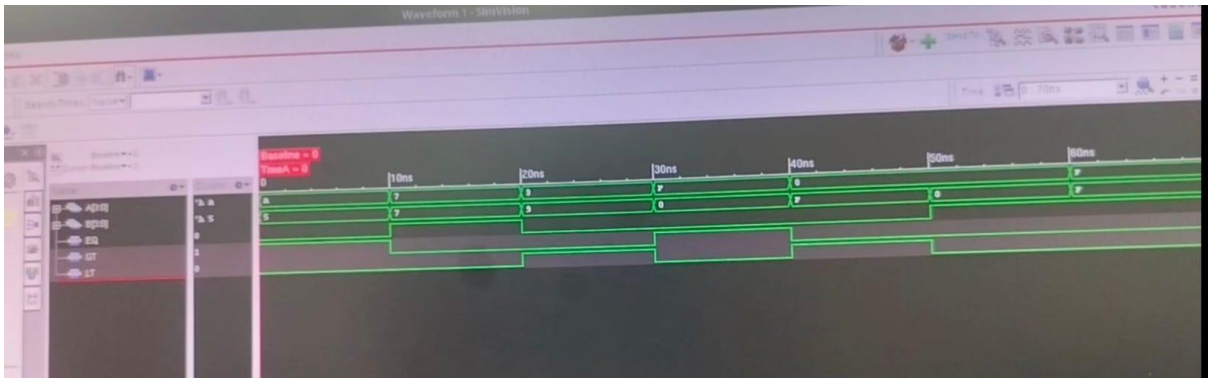
        // Test case 7: Both inputs are maximum
        A = 4'd15; B = 4'd15;
        #10;

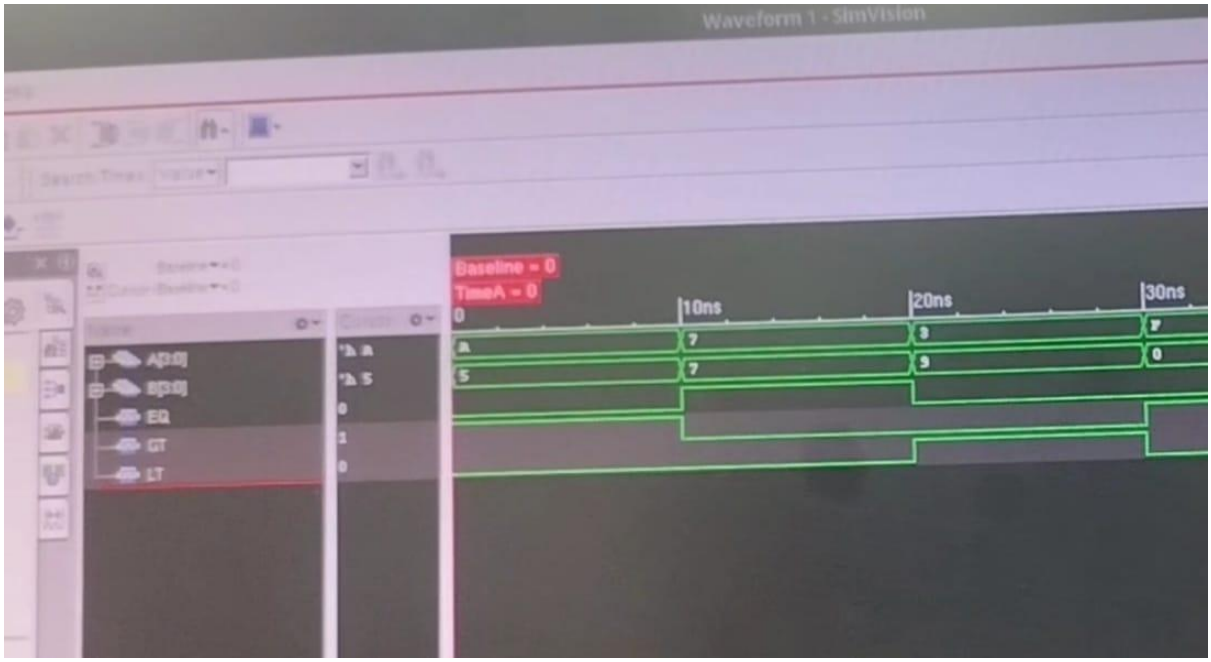
        $display("Testbench completed.");
        $finish;
    end

    initial begin
$dumpfile("compare.vcd");
$dumpvars;     end

endmodule
```

Simulation Results:





Experiment 2(Sequential Circuit) -Ring Counter

Procedure:

1. Create folders with names “RTL” , “TB” and “RESULT” to store the code, testbench and simulation results respectively.
2. Open the terminal.

3. Open the folder RTL using cd command.
4. Use “gedit ring_counter.v” to open and edit a Verilog file, add your code her and save it.
5. To check for errors, close the code file and in the terminal type “ncverilog +access+r ring_counter.v”.
6. To open folder “TB” type “ cd ../TB”.
7. And type “gedit ring_counter_tb.v” to open and edit a Verilog file, add your testbench code here.
8. Close the testbench code file and type “cd ../RESULT” to open the result folder.
9. To execute the tetsbench and code type “ncverilog +access+r ../RTL/ring_counter.v ../TB/ring_counter_tb.v”.
10. This generates the vcd file in RESULT folder.
11. To check if the vcd file is present in RESULT folder enter “ls”.
12. For simulation we have used simvision.
13. To open the simulation enter simvision counter.vcd

Code:

```
module ring_counter (
input clk,     input
reset,
   output reg [3:0] q
);

always @(posedge clk or posedge reset) begin
   if (reset)        q
<= 4'b0001;
   else        q <= {q[2:0],
q[3]};
end

endmodule
```

Testbench Code:

```
module ring_counter_tb;

   reg clk;
reg reset;
   wire [3:0] q;

   ring_counter uut (
      .clk(clk),
      .reset(reset),
      .q(q)
   );

   initial begin         clk =
0;         forever #5 clk =
~clk;     end

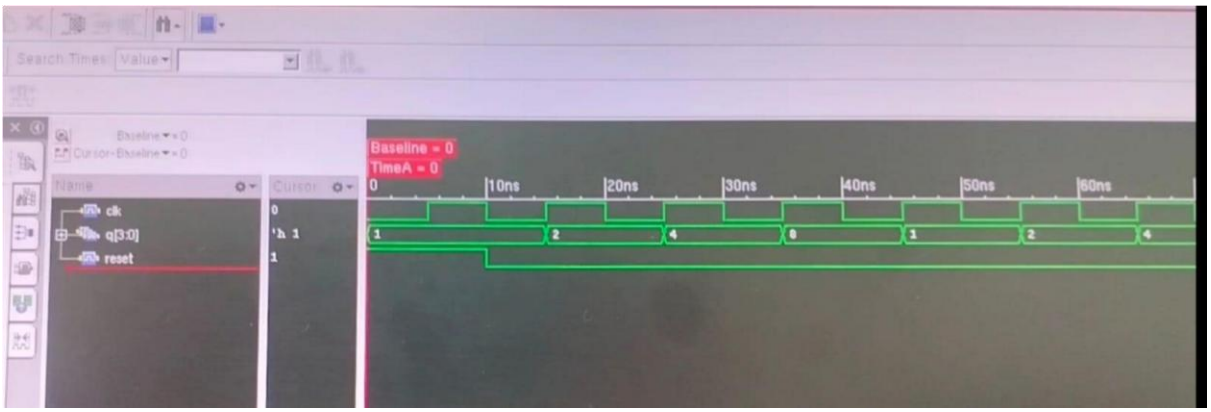
   initial begin
reset = 1;         #10
reset = 0;
#100 $finish;
end

   initial begin
      $monitor("Time=%0t reset=%b q=%b", $time, reset, q);
end

   initial begin
      $dumpfile("counter.vcd");
      $dumpvars;
   end

endmodule
```

Simulation Results:


Search Times:
Value
Baseline = 0
TimeA = 0
0
10ns
20ns
30ns
40ns
50ns
60ns
clk
q[3:0]
reset
'h 1
1
2
4
0
1
2
4